

RENOWA REPRESENTAÇÕES

Sistema de Gestão Comercial Interno

DOCUMENTO DE ARQUITETURA TÉCNICA

Versão 1.2 | Fevereiro 2026

Desenvolvimento Solo | Confidencial

1. Visão Geral do Projeto

O Sistema Renowa é um aplicativo interno de gestão comercial voltado exclusivamente para a equipe da Renowa Representações. O sistema opera em duas plataformas — Web e Mobile — que se comunicam por meio de uma API central, com suporte a funcionamento offline no aplicativo móvel.

1.1 Objetivos Principais

- Centralizar o cadastro de clientes, pedidos, produtos e transportadoras
- Permitir emissão de orçamentos em PDF com layout profissional
- Funcionar offline no mobile, sincronizando ao recuperar conexão
- Oferecer painel financeiro com fluxo de caixa, comissões e inadimplência
- Rastrear vendas por vendedor/colaborador

1.2 Plataformas

Plataforma	Tecnologia	Observação
Web (navegador)	React + Vite + Tailwind	Interface completa de gestão
Mobile (app)	React Native + Expo	Foco em Pedidos com suporte offline
Backend (API)	Node.js + NestJS	Lógica de negócio e sincronização
Banco de Dados	PostgreSQL (Railway)	Dados relacionais centralizados

2. Stack Tecnológica Definida

2.1 Backend

Tecnologia	Papel	Justificativa
Node.js	Runtime do servidor	Ecossistema JS unificado no projeto
NestJS	Framework principal	Arquitetura modular, escalável, com injeção de dependência
PostgreSQL	Banco de dados	Relacional, ideal para os relacionamentos do sistema
Railway	Hospedagem (DB + API)	Simples, barato, sem lock-in, controle total

2.2 Frontend Web

Tecnologia	Papel	Justificativa
React + Vite	Framework + bundler	Performance, produtividade, ecossistema maduro
Tailwind CSS	Estilização	Utilitário, rápido de customizar com as cores da Renowa
shadcn/ui	Componentes UI	Visual limpo, alinhado ao mockup aprovado
Recharts	Gráficos	Dashboard financeiro (Evolução de Vendas, Curva ABC)
@react-pdf/renderer	Geração de PDF	Orçamentos com logo e layout limpo

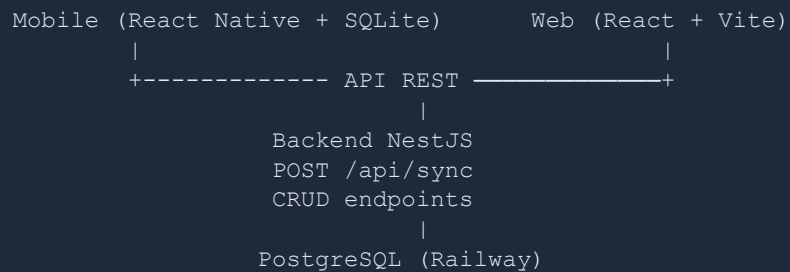
2.3 Mobile

Tecnologia	Papel	Justificativa
React Native	Framework mobile	Compartilha lógica com o Web (JavaScript)
Expo	Toolchain mobile	Facilita build e deploy para dev solo
expo-sqlite	Storage offline	Banco local para pedidos sem internet
NetInfo	Deteção de rede	Trigger para acionar a sincronização

3. Arquitetura do Sistema

3.1 Fluxo Geral

O sistema segue uma arquitetura de três camadas, onde Web e Mobile nunca acessam o banco diretamente — toda a comunicação passa pela API NestJS.



3.2 Estrutura de Pastas do Projeto

```
renowa/
  backend/          ← NestJS
    src/
      clients/      ← Módulo Clientes
      orders/       ← Módulo Pedidos
      products/     ← Módulo Produtos
      finance/      ← Módulo Financeiro
      transport/    ← Módulo Transportadoras
      sync/         ← Módulo crítico de sincronização offline
      auth/         ← Autenticação e controle de acesso

  web/              ← React + Vite
    src/
      pages/        ← Clientes, Pedidos, Financeiro...
      components/   ← Sidebar, Header, Tabelas, Forms

  mobile/           ← React Native + Expo
    src/
      screens/      ← Telas do app
      storage/      ← SQLite + sync_queue
```

4. Estratégia de Sincronização Offline

4.1 Princípio

O app mobile opera de forma totalmente independente quando não há internet. Ao recuperar a conexão, envia os dados pendentes ao backend e recebe atualizações do servidor. O sync é bidirecional.

4.2 Sync Bidirecional

Direção	O que é sincronizado
Mobile → Servidor	Clientes novos, Pedidos novos, atualizações feitas offline
Servidor → Mobile	Produtos, Transportadoras, Clientes atualizados por outros usuários

4.3 Regra Crítica — UUID

⚠ REGRA OBRIGATÓRIA: Nunca usar ID sequencial no mobile offline.

Problema: Dois usuários criam o Pedido #101 offline ao mesmo tempo → conflito.

Solução: Mobile gera UUID (campo 'uuid') → Servidor salva e retorna ID sequencial oficial.
O campo se chama 'uuid' (não uuid_mobile) — pode ser usado por qualquer plataforma.

Exemplo:

Qualquer cliente cria: uuid = 'a82f-92aa-...'

Servidor salva: id = 105 (ID oficial sequencial)

4.4 Estratégia de Conflito

Regra definida: Último updated_at vence (Last Write Wins).

Exemplo:

Mobile atualizou cliente às 10:00 → updated_at = 10:00

Web atualizou o mesmo cliente às 10:05 → updated_at = 10:05

Resultado: versão da Web prevalece (10:05 > 10:00)

O servidor sempre compara o updated_at recebido com o do banco.

Se o banco tiver updated_at mais recente, descarta a atualização do mobile.

4.5 last_sync_timestamp

O mobile armazena localmente o timestamp da última sincronização bem-sucedida.

Uso: GET /api/sync?since=2026-02-25T18:00:00

Benefício: O servidor retorna apenas os registros modificados após esse timestamp. Isso reduz drasticamente o tráfego de dados — especialmente importante em conexões móveis.

Fluxo:

1. App envia: GET /sync?since=last_sync_timestamp
2. Servidor retorna apenas deltas (registros com updated_at > since)
3. App atualiza last_sync_timestamp = agora

4.6 Soft Delete — deleted_at

Registros nunca são apagados fisicamente do banco — apenas marcados com deleted_at.

Por quê:

- Mobile pode excluir um registro offline
- O servidor (ou outro usuário) ainda pode estar usando esse registro
- O sync precisa saber que o registro foi removido — não apenas que sumiu

Regra: deleted_at NULL = ativo | deleted_at preenchido = excluído

A consulta GET /sync?since=... inclui registros com deleted_at recente, para que o mobile saiba quais itens remover do SQLite local.

4.7 Fluxo de Sincronização

1. Usuário cria/edita/exclui registro sem internet
 - Salva no SQLite local (com uuid gerado localmente)
 - Entra na sync_queue local (SQLite only — não existe no PostgreSQL)
2. App detecta internet (NetInfo.isConnected === true)
 - Percorre a sync_queue
 - Envia POST /api/sync com os itens pendentes
3. Servidor processa (regra: último updated_at vence)
 - Salva no PostgreSQL
 - Retorna IDs oficiais e registros deletados
4. App atualiza SQLite local
 - Substitui uuid pelo ID oficial
 - Aplica soft deletes recebidos
 - Limpa sync_queue
 - Atualiza last_sync_timestamp
5. App busca atualizações do servidor
 - GET /api/sync?since=last_sync_timestamp

- Recebe apenas deltas: Produtos, Transportadoras, Clientes
- Atualiza SQLite local

5. Módulos do Sistema

5.1 Prioridade de Desenvolvimento

#	Módulo	Observação
1º	Clientes	CRUD completo — base de todos os outros módulos
2º	Pedidos	Coração do sistema + geração de PDF
3º	Produtos	Catálogo por fornecedor para agilizar pedidos
4º	Transporte	Vinculado a Clientes e Pedidos
5º	Financeiro	Fluxo de caixa, comissões, inadimplência

5.2 Integrações Externas

Integração	Status	Observação
Sintegra	Fase 2 (após CRUD estável)	Preenchimento automático de dados de cliente via CNPJ — ~R\$30/mês
PDF Orçamento	Fase 1 (junto com Pedidos)	Tratar como mini-projeto separado — requer atenção ao layout

6. Modelo de Dados (Visão Geral)

6.1 Tabelas Core

Tabela	Campos Principais
usuarios	id, uuid, email, nome, senha_hash, role (admin/vendedor), created_at, updated_at, deleted_at
clientes	id, uuid, razao_social, cnpj, email, tel, endereco, bairro, cidade, uf, cep, transportadora_id, pgd_padrao, created_at, updated_at, deleted_at
transportadoras	id, uuid, razao_social, cnpj, telefone, endereco_completo, created_at, updated_at, deleted_at
produtos	id, uuid, fornecedor_id, codigo, descricao, preco_base, created_at, updated_at, deleted_at
fornecedores	id, uuid, razao_social, cnpj, created_at, updated_at, deleted_at

6.2 Tabelas de Movimentação

Tabela	Campos Principais
pedidos	id, uuid, numero_pedido, cliente_id, vendedor_id, fornecedor_id, data, status, total_sem_imposto, total_com_imposto, created_at, updated_at, deleted_at
itens_pedido	id, uuid, pedido_id, produto_id, qtd_caixas, qtd_unitaria, preco_unitario, desconto_perc, total_item, created_at, updated_at, deleted_at

6.3 Tabelas Financeiras

Tabela	Campos Principais
financeiro_movimentacao	id, uuid, tipo (Custo Fixo/Rotativo/Venda), valor, data, descricao, created_at, updated_at, deleted_at
comissoes	id, uuid, pedido_id, nfe, valor_faturado, perc_comissao, valor_comissao, data_faturamento, created_at, updated_at
inadimplencia	id, uuid, cliente_id, empresa_devedora, valor_aberto, observacao, created_at, updated_at, deleted_at

6.4 Sync — SQLite Mobile (local only)

Tabela	Descrição
sync_queue	Existe APENAS no SQLite local do mobile. Campos: id, entidade, uuid, payload (JSON), status (pendente/enviado), created_at. Nunca replicada no PostgreSQL.

Tabela	Descrição
sync_meta	Guarda last_sync_timestamp localmente. Usado no GET /api/sync?since=timestamp para buscar apenas deltas do servidor.

i Regras de banco — todas as tabelas do PostgreSQL devem ter:

- uuid UUID UNIQUE → gerado pelo cliente (web ou mobile), nunca repetível
- created_at TIMESTAMP DEFAULT now() → preenchido automaticamente na inserção
- updated_at TIMESTAMP DEFAULT now() → atualizado por trigger a cada UPDATE
- deleted_at TIMESTAMP NULL → soft delete (NULL = ativo)

6.5 Índices Obrigatórios

A rota GET /api/sync?since=timestamp faz queries por updated_at em todas as tabelas. Sem índices, o banco faz full scan e a sincronização fica lenta conforme o volume cresce.

```
-- Obrigatório em todas as tabelas sincronizáveis:
CREATE INDEX idx_<tabela>_uuid          ON <tabela>(uuid);
CREATE INDEX idx_<tabela>_updated_at    ON <tabela>(updated_at);
CREATE INDEX idx_<tabela>_deleted_at    ON <tabela>(deleted_at);

-- Exemplo para a tabela pedidos:
CREATE INDEX idx_pedidos_uuid          ON pedidos(uuid);
CREATE INDEX idx_pedidos_updated_at    ON pedidos(updated_at);
CREATE INDEX idx_pedidos_deleted_at    ON pedidos(deleted_at);
```

6.6 Trigger — updated_at Automático

Para garantir que updated_at seja sempre atualizado em qualquer UPDATE, independente do backend lembrar de fazê-lo, utilizar um trigger no PostgreSQL.

```
-- Função reutilizável (criar uma vez):
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Aplicar em cada tabela:
CREATE TRIGGER trg_<tabela>_updated_at
BEFORE UPDATE ON <tabela>
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- Exemplo para clientes:
```

```
CREATE TRIGGER trg_clientes_updated_at
BEFORE UPDATE ON clientes
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

6.7 Regra — numero_pedido

⚠ numero_pedido é gerado EXCLUSIVAMENTE pelo servidor. Nunca pelo mobile.

Por quê:

- Mobile offline não tem como saber qual é o próximo número da sequência
- Dois mobiles offline gerariam o mesmo número → conflito

Fluxo correto:

1. Mobile cria pedido offline → armazena apenas uuid (sem numero_pedido)
2. Sync enviado ao servidor
3. Servidor gera numero_pedido sequencial e retorna ao mobile
4. Mobile atualiza o registro local com o numero_pedido recebido

No SQLite local, numero_pedido pode ficar NULL até a sincronização ocorrer.

7. Ordem de Desenvolvimento

Fase	Entregável	Detalhe
1	Modelagem do Banco	Scripts SQL com todas as tabelas, campos e relacionamentos
2	Backend NestJS	Módulos Core + rota POST /api/sync + autenticação
3	Web — Clientes e Pedidos	Módulos prioritários com CRUD completo
4	PDF do Orçamento	Mini-projeto dedicado — layout limpo com logo
5	Web — demais módulos	Produtos, Transporte, Financeiro
6	Mobile (React Native)	Telas de Pedidos com SQLite offline + sync
7	Sintegra	Integração de busca por CNPJ — somente após CRUD estável

8. Alertas e Regras Críticas

8.1 Regras de Arquitetura

- NUNCA conectar Frontend diretamente ao banco — sempre via API
- NUNCA usar ID sequencial em registros criados offline — usar UUID UNIQUE (vale para web e mobile)
- NUNCA gerar numero_pedido no mobile — gerado exclusivamente pelo servidor após sync
- SEMPRE incluir created_at DEFAULT now(), updated_at DEFAULT now() e deleted_at em toda tabela
- Trigger set_updated_at obrigatório em todas as tabelas — updated_at nunca manual
- Índices obrigatórios em uuid, updated_at e deleted_at — essencial para performance do sync
- Soft delete obrigatório — deleted_at NULL = ativo, deleted_at preenchido = excluído
- sync_queue e sync_meta existem APENAS no SQLite local do mobile — nunca no PostgreSQL
- Sync usa last_sync_timestamp: GET /api/sync?since=timestamp — trafegar apenas deltas
- Conflito: último updated_at vence (Last Write Wins) — servidor decide
- Sync é bidirecional — mobile envia e recebe dados

8.2 Pontos de Atenção no Desenvolvimento

⚠ PDF — Mais trabalhoso do que parece

O requisito 'sem linhas divisórias, layout limpo, com logo' exige tempo dedicado.

Tratar a geração de PDF como um mini-projeto separado dentro da Fase 4.

⚠ Sync Offline — A parte mais complexa do sistema

Desenvolver com calma. Testar cenários: sem internet, internet voltando, conflitos.

⚠ Sintegra — Implementar por último

Somente após o CRUD de Clientes estar 100% funcional e testado.

8.3 Identidade Visual

Elemento	Valor	Uso
Cor primária	#2A9D8F (Verde Teal)	Botões principais, destaques, sidebar
Cor secundária	#FFFFFF (Branco)	Texto sobre botões e fundo escuro
Fundo da aplicação	#F4F7F6 (Off-white)	Background geral das páginas
Tipografia	Sans-serif (Arial/Inter)	Todo o sistema

